Unit IV Class Activity

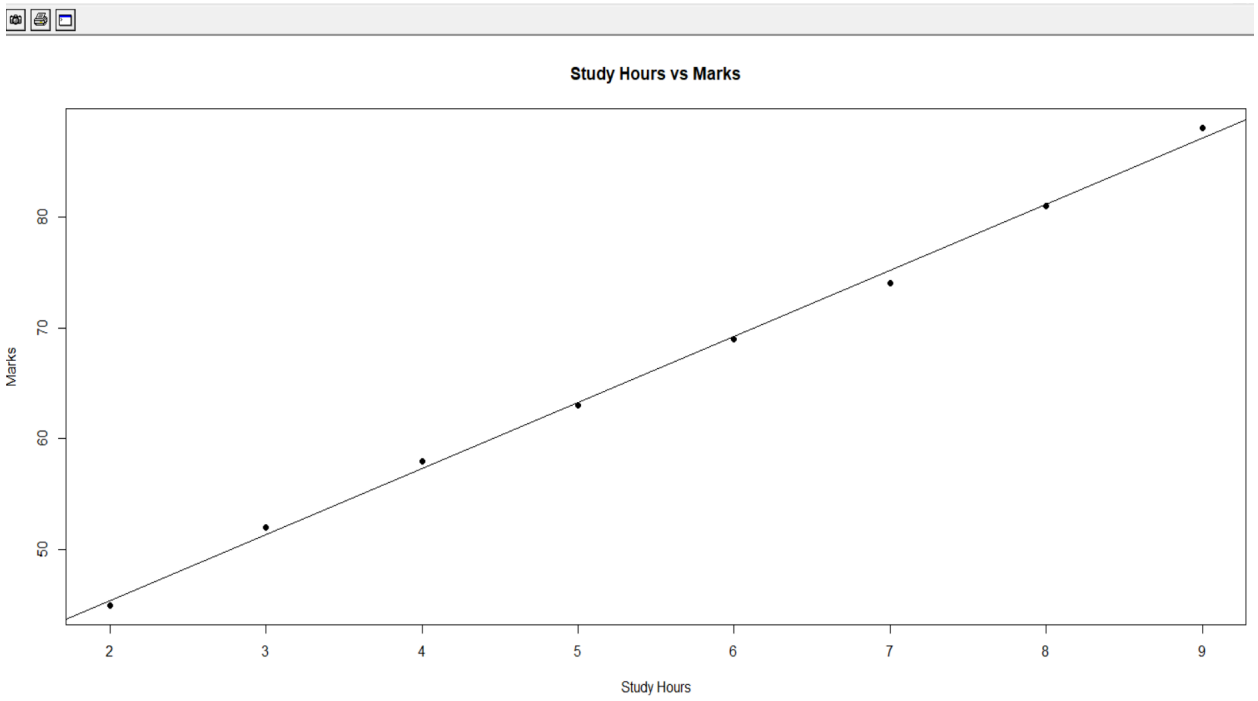
Date:14-08-26

1.To determine whether students who spend more hours studying obtain higher marks.

R Code:

```
data <- data.frame(
  StudyHours = c(2, 3, 4, 5, 6, 7, 8, 9),
  Marks = c(45, 52, 58, 63, 69, 74, 81, 88)
)
print(data)
plot(
  data$StudyHours,
  data$Marks,
  main = "Study Hours vs Marks",
  xlab = "Study Hours",
  ylab = "Marks",
  pch = 19
)
abline(
  lm(Marks ~ StudyHours, data = data)
)
```

Output:



Interpretation

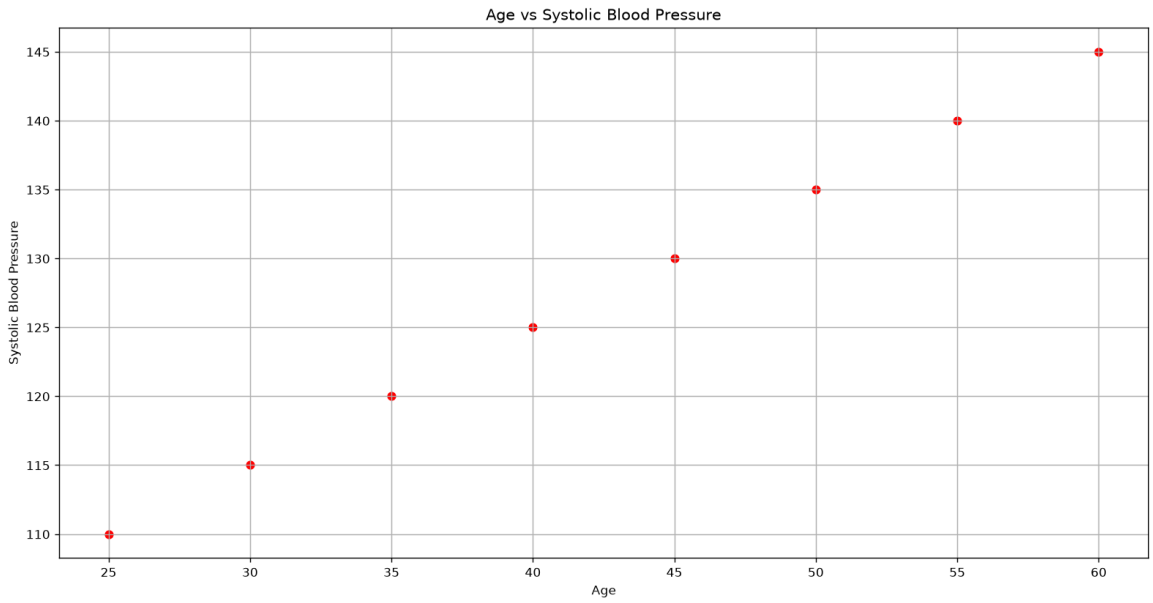
The scatterplot shows a positive correlation between study hours and marks. As the number of study hours increases, students' marks also increase. Therefore, students who spend more time studying tend to obtain higher marks in this dataset.

2. A hospital records patients' age and systolic blood pressure.

Python Code:

```
import matplotlib.pyplot as plt

age = [25, 30, 35, 40, 45, 50, 55, 60]

blood_pressure = [110, 115, 120, 125, 130, 135, 140, 145]

plt.scatter(age, blood_pressure, color="red")

plt.xlabel("Age")

plt.ylabel("Systolic Blood Pressure")

plt.title("Age vs Systolic Blood Pressure")

plt.grid(True)

plt.show()
```

Output:



Interpretation

The scatterplot shows a positive relationship between age and systolic blood pressure. As age increases, systolic blood pressure also tends to increase. Therefore, older patients generally show higher systolic blood pressure in this dataset.

3.Develop a scatterplot and determine whether sales increase with advertising expenditure.

Part 1 — Advertising Expenditure vs Monthly Sales

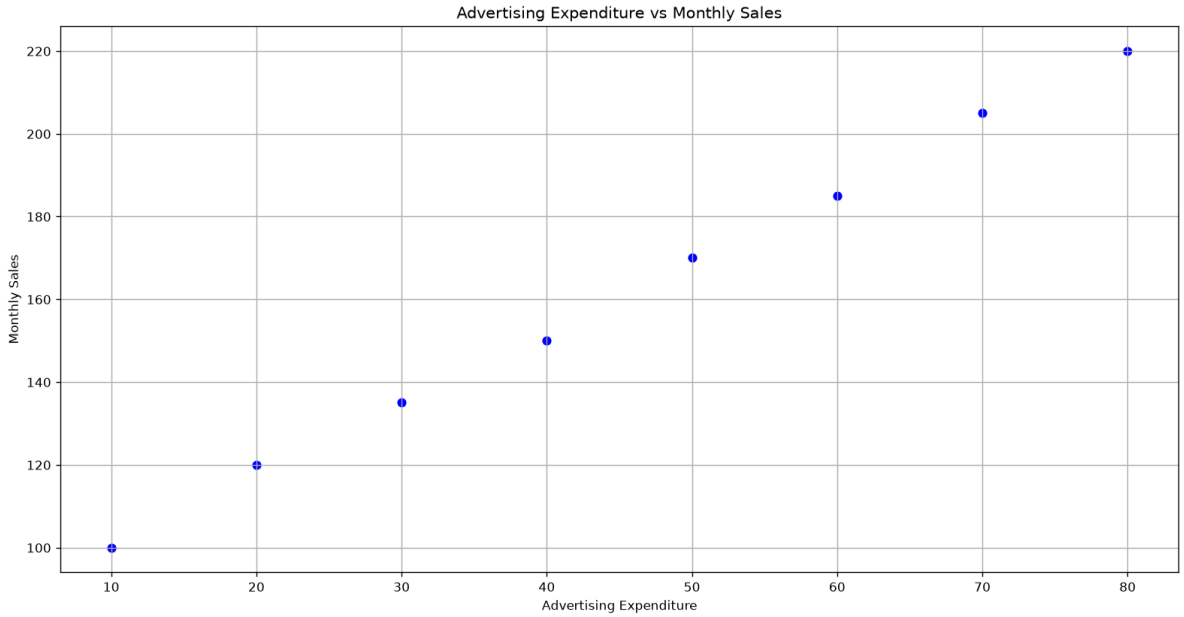
Python code:

```
import matplotlib.pyplot as plt

advertising = [10, 20, 30, 40, 50, 60, 70, 80]

sales = [100, 120, 135, 150, 170, 185, 205, 220]

plt.scatter(advertising, sales, color="blue")

plt.xlabel("Advertising Expenditure")

plt.ylabel("Monthly Sales")

plt.title("Advertising Expenditure vs Monthly Sales")
```

plt.grid(True)

plt.show()

Output:



Interpretation

The scatterplot shows a positive relationship between advertising expenditure and monthly sales. As advertising expenditure increases, monthly sales also increase. Therefore, the data suggests that higher advertising expenditure is associated with higher sales.

Part 2 — Correlation Matrix and Correlogram

R code:

data <- data.frame(

Income = c(30, 40, 50, 60, 70),

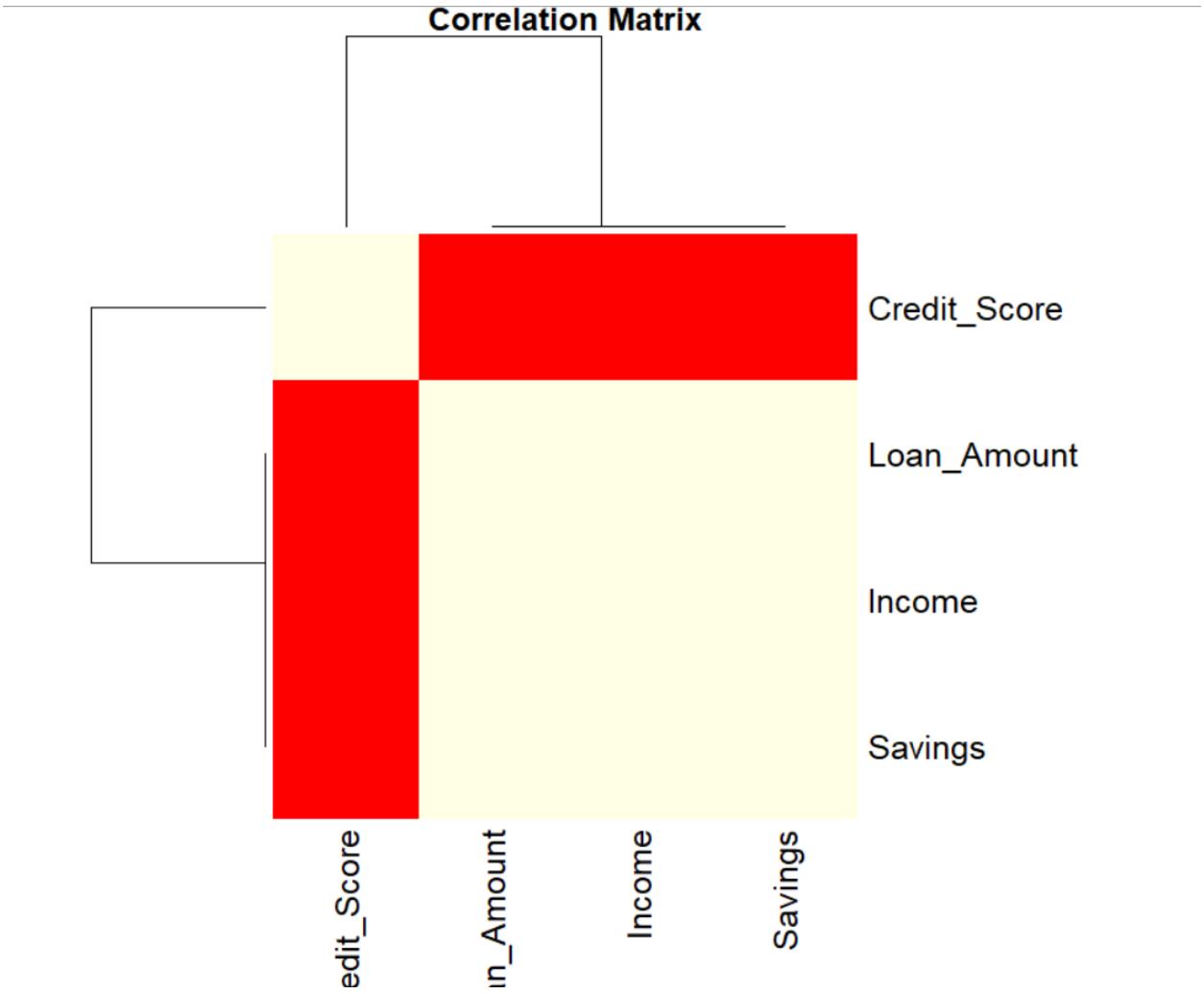
Savings = c(5, 10, 15, 20, 25),

Loan_Amount = c(20, 25, 30, 35, 40),

Credit_Score = c(650, 680, 700, 730, 760)

```
)
cor_matrix <- cor(data)
print(cor_matrix)
heatmap(
  cor_matrix,
  main = "Correlation Matrix",
  symm = TRUE,
  col = heat.colors(20),
  margins = c(8, 8)
)
```

Output:



Interpretation

The correlation matrix shows the strength and direction of relationships between:

- Income
- Savings
- Loan Amount
- Credit Score

For this sample dataset, the variables have strong positive correlations because they were constructed to increase together.

The strongest positive correlations are essentially:

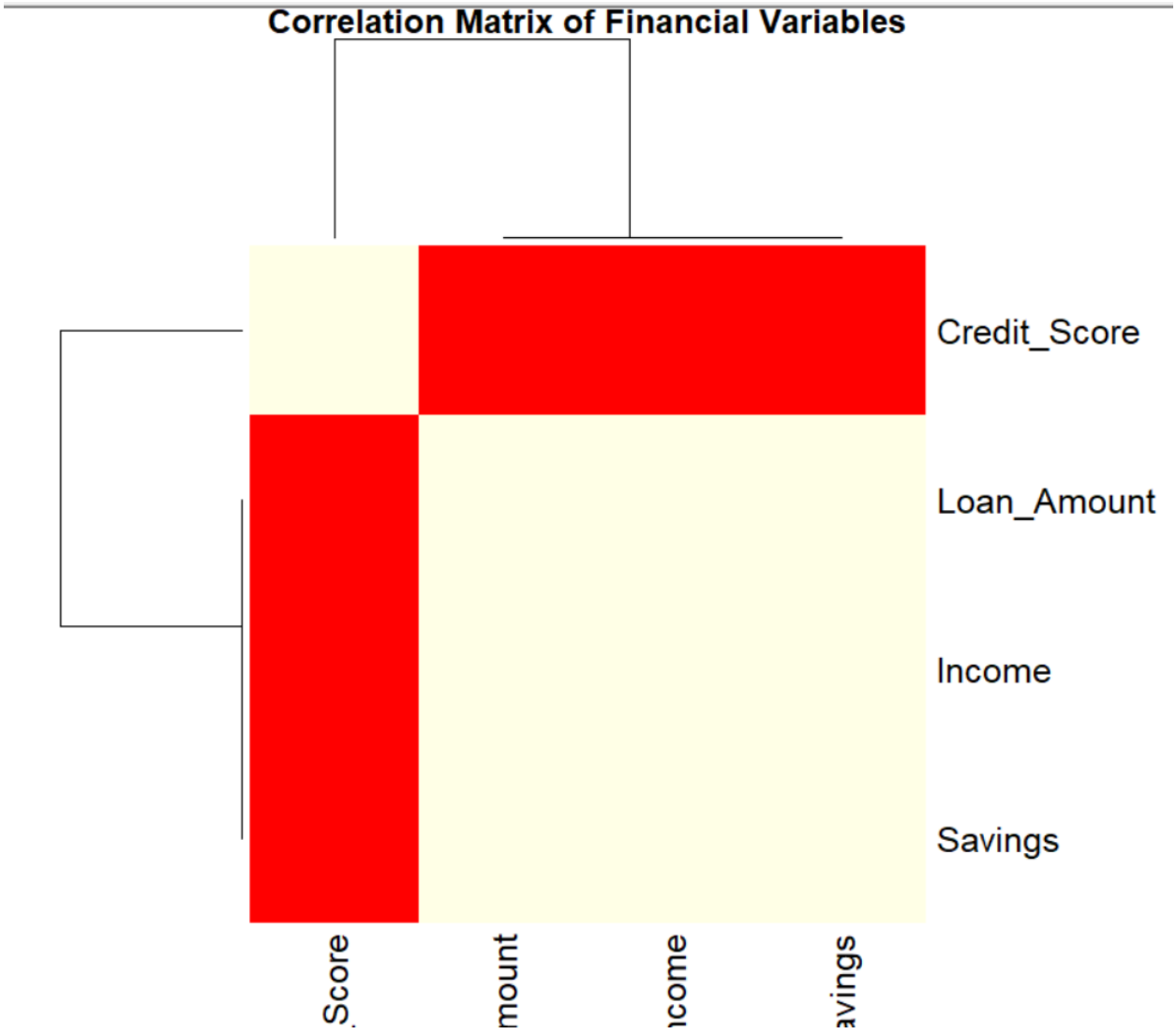
Income and Savings, Income and Loan Amount, and Income and Credit Score.

4.Correlation Matrix and Correlogram of Financial Variables

R Program:

```
data <- data.frame(
  Income = c(30, 40, 50, 60, 70),
  Savings = c(5, 10, 15, 20, 25),
  Loan_Amount = c(20, 25, 30, 35, 40),
  Credit_Score = c(650, 680, 700, 730, 760)
)
cor_matrix <- cor(data)
print(cor_matrix)
heatmap(
  cor_matrix,
  main = "Correlation Matrix of Financial Variables",
  symm = TRUE,
  col = heat.colors(20)
)
```

Output:



Interpretation

The correlation matrix shows the relationships among Income, Savings, Loan Amount, and Credit Score.

For this dataset, the variables show strong positive correlations because all four variables generally increase together.

Answer:

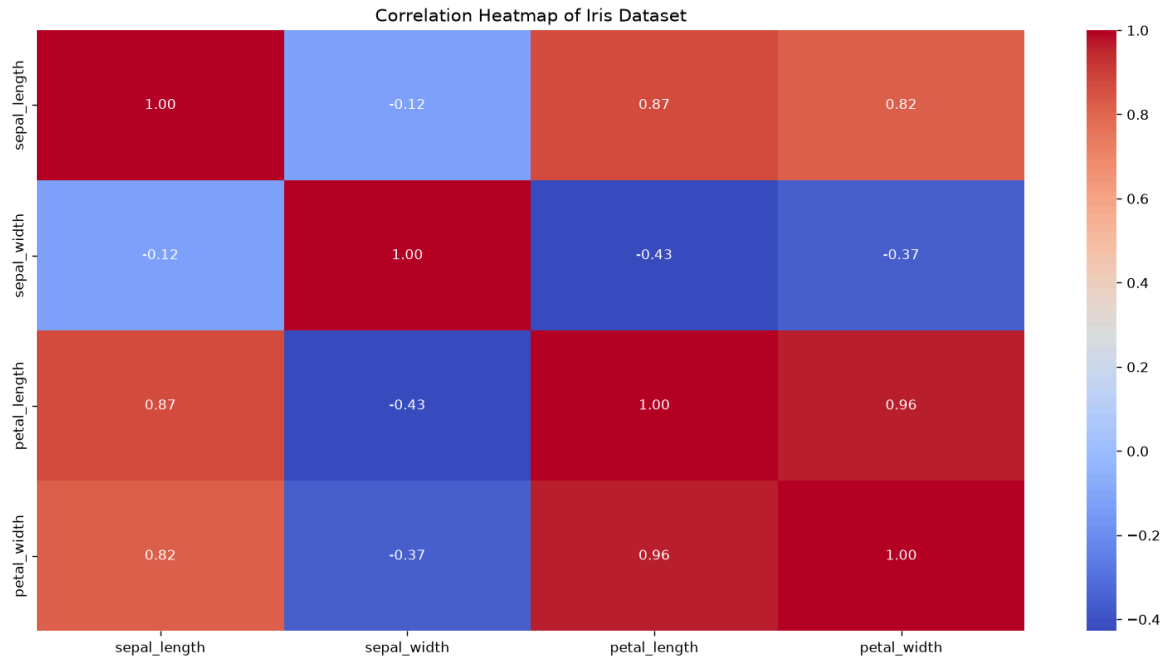
> Income, Savings, Loan Amount, and Credit Score exhibit strong positive correlations with each other. The correlogram provides a visual representation of these relationships, where stronger correlations are represented by more intense colors.

5. Correlation Matrix and Heatmap of Iris Dataset

Python Program:

```
import seaborn as sns

import matplotlib.pyplot as plt

iris = sns.load_dataset("iris")

data = iris[["sepal_length", "sepal_width", "petal_length", "petal_width"]]

print("Correlation Matrix:")

print(correlation)

plt.figure(figsize=(8, 6))

sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")

plt.title("Correlation Heatmap of Iris Dataset")

plt.show()
```

Output:



Negative Correlations

The Iris dataset has these main negative correlations:

- Sepal Length and Sepal Width → negative correlation
- Sepal Width and Petal Length → negative correlation
- Sepal Width and Petal Width → negative correlation

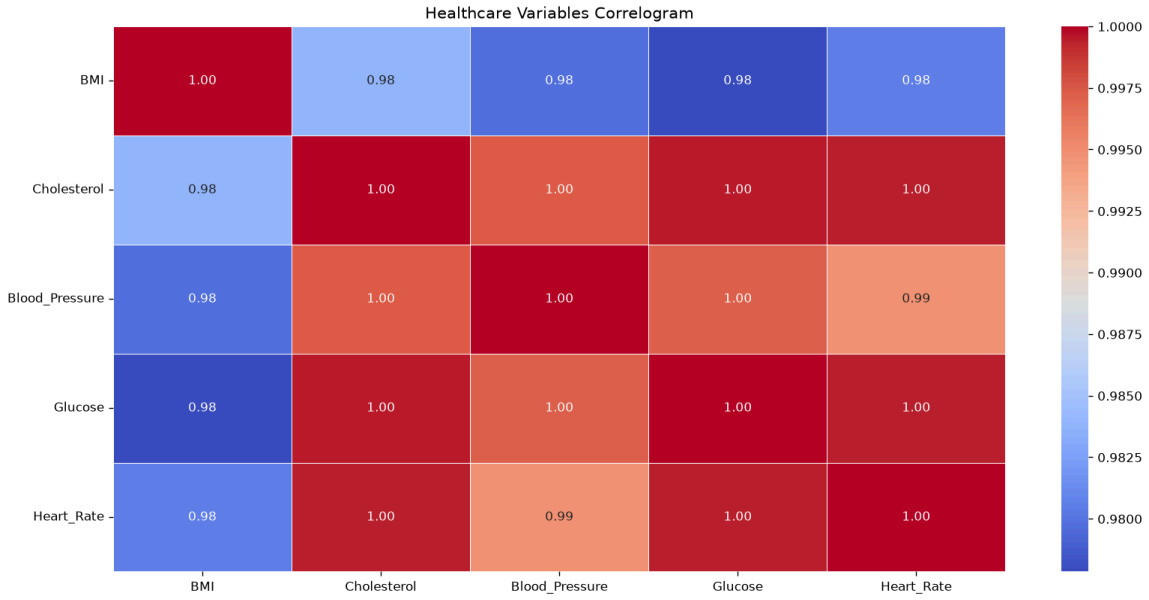
6.Correlogram and Multicollinearity

Python Program:

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = pd.DataFrame({

    "BMI": [22, 25, 28, 30, 32],

    "Cholesterol": [180, 195, 210, 230, 250],

    "Blood_Pressure": [115, 120, 130, 140, 150],

    "Glucose": [85, 95, 105, 120, 135],

    "Heart_Rate": [68, 72, 75, 80, 85]

})

cor_matrix = data.corr()

print("Correlation Matrix:")

print(cor_matrix)

plt.figure(figsize=(8, 6))

sns.heatmap(

    cor_matrix,

    annot=True,
```

```
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)
plt.title("Healthcare Variables Correlogram")
plt.show()
```

Output:



Interpretation

> The correlogram shows the correlation between BMI, Cholesterol, Blood Pressure, Glucose, and Heart Rate. Strong correlations between variables indicate possible multicollinearity. In this sample dataset, the health indicators generally increase together, so several variables show strong positive relationships.

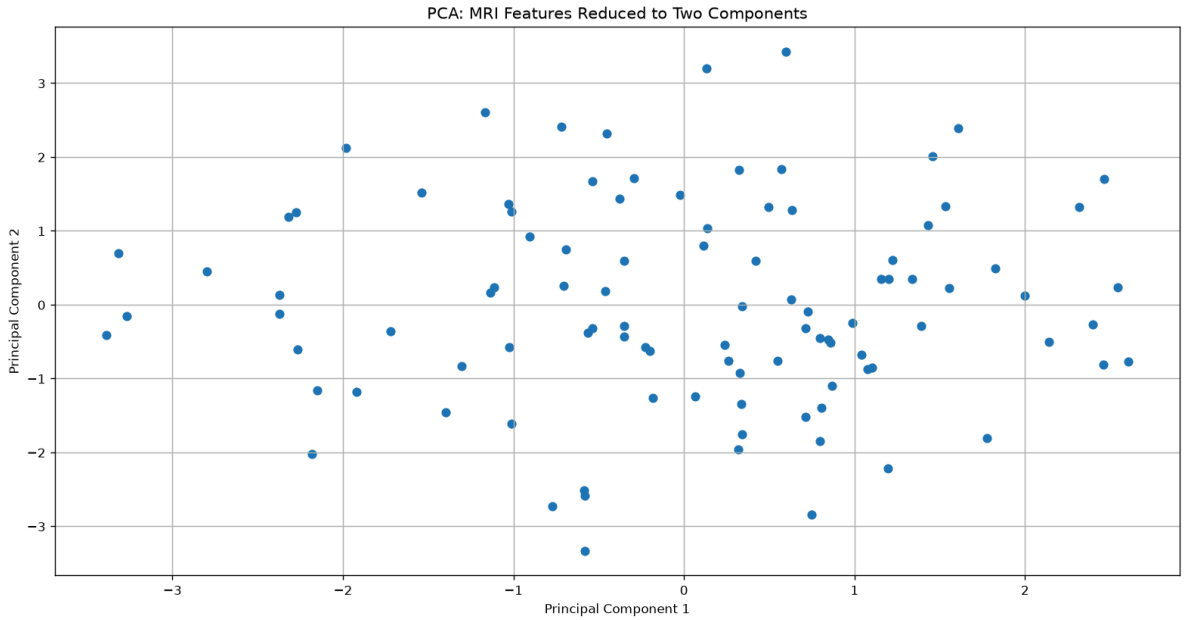
Multicollinearity: When two or more predictor variables are highly correlated with each other, they may provide overlapping information. This can make statistical or machine-learning models less reliable.

7. PCA for Dimensionality Reduction

Python Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
np.random.seed(10)
data = np.random.rand(100, 25)
scaled_data = StandardScaler().fit_transform(data)
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
plt.figure(figsize=(8, 6))
plt.scatter(pca_data[:, 0], pca_data[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA: MRI Features Reduced to Two Components")
plt.grid(True)
plt.show()
```

Output:



Interpretation

PCA reduces the original 25 MRI features to 2 principal components while retaining the maximum possible variation in the data. The scatter plot shows the transformed observations in a two-dimensional space.

8. PCA on Iris Dataset

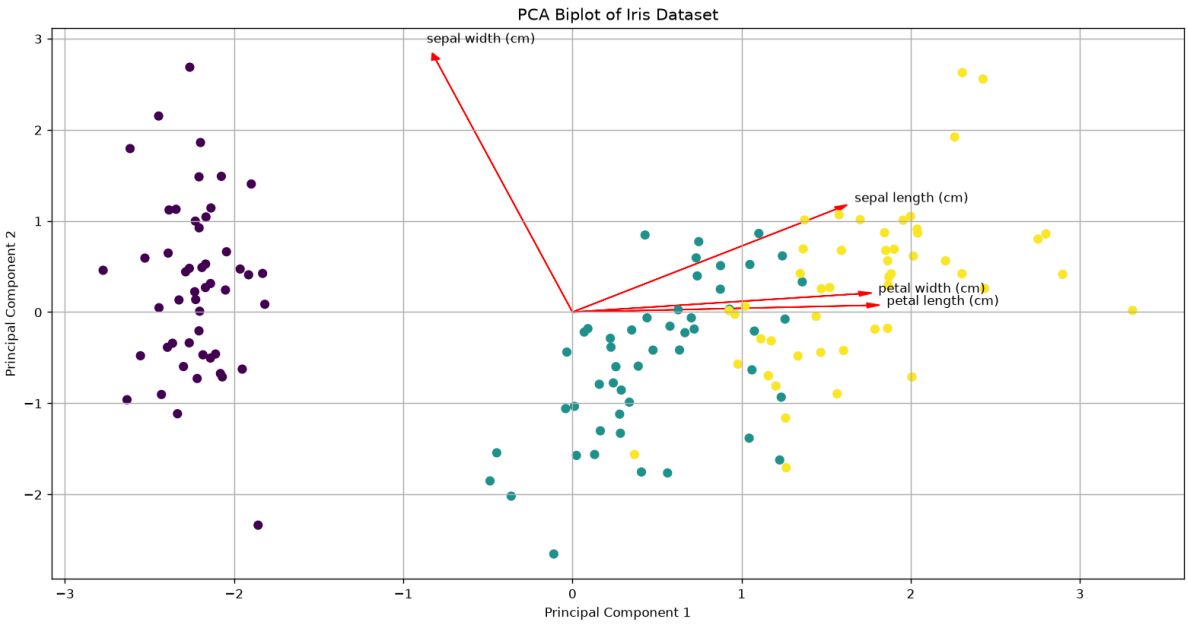
Python Program:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

iris = load_iris()

X = iris.data

y = iris.target
```

```
X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
print("\nCumulative Explained Variance:")
print(pca.explained_variance_ratio_.cumsum())
print("\nPrincipal Components:")
print(pca.components_)
pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)
plt.figure(figsize=(9, 7))
for i, feature in enumerate(iris.feature_names):
    plt.arrow(
        0, 0,
        pca.components_[0, i] * 3,
        pca.components_[1, i] * 3,
        color="red",
        head_width=0.05
    )
    plt.text(
```

```
        pca.components_[0, i] * 3.2,
        pca.components_[1, i] * 3.2,
        feature,
        fontsize=10
    )
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Biplot of Iris Dataset")
plt.grid(True)
plt.show()
```

Output:



Interpretation

- PC1 explains the highest variance in the Iris dataset.
- PC1 captures the largest amount of information from the original four variables.
- PC2 explains the second-highest variance.
- The biplot shows how the original Iris features contribute to PC1 and PC2.

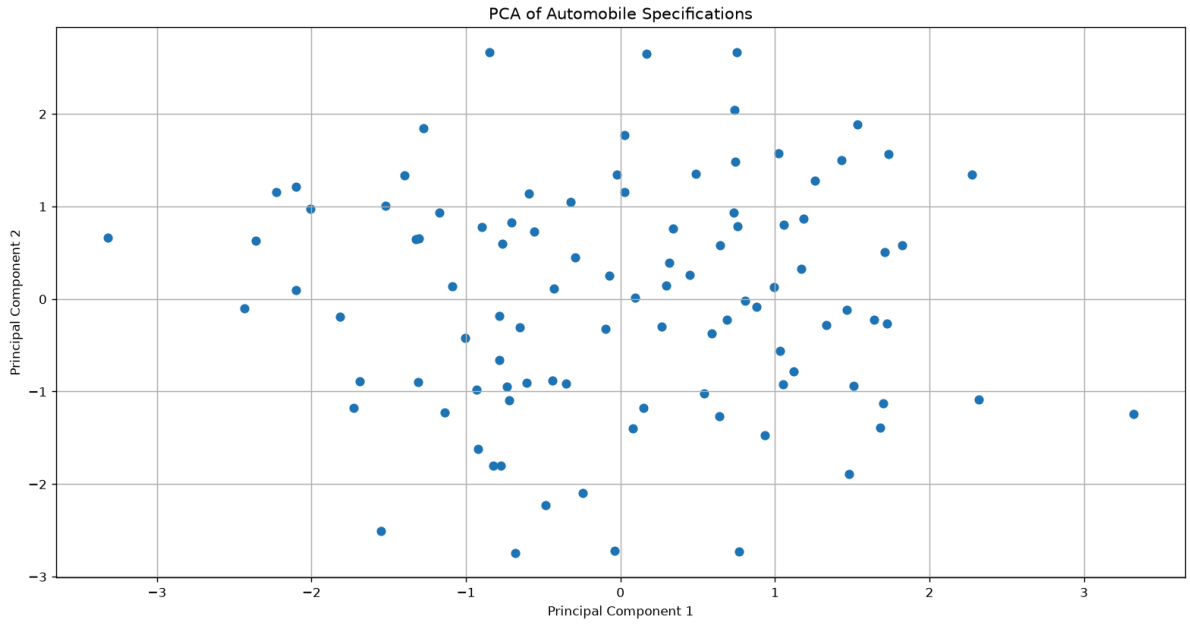
9. PCA on Automobile Specifications

Python Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

np.random.seed(10)

data = np.random.rand(100, 15)

scaled_data = StandardScaler().fit_transform(data)

pca = PCA(n_components=2)
```

```
pca_data = pca.fit_transform(scaled_data)

print("Explained Variance Ratio:")

print(pca.explained_variance_ratio_)

print("\nTotal Variance Explained:")

print(pca.explained_variance_ratio_.sum())

plt.figure(figsize=(8, 6))

plt.scatter(

    pca_data[:, 0],

    pca_data[:, 1]

)

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("PCA of Automobile Specifications")

plt.grid(True)

plt.show()
```

Output:



Interpretation

The original automobile dataset contains 15 specifications. PCA reduces these 15 dimensions to two principal components (PC1 and PC2). The scatter plot visualizes the automobiles in this reduced two-dimensional space.

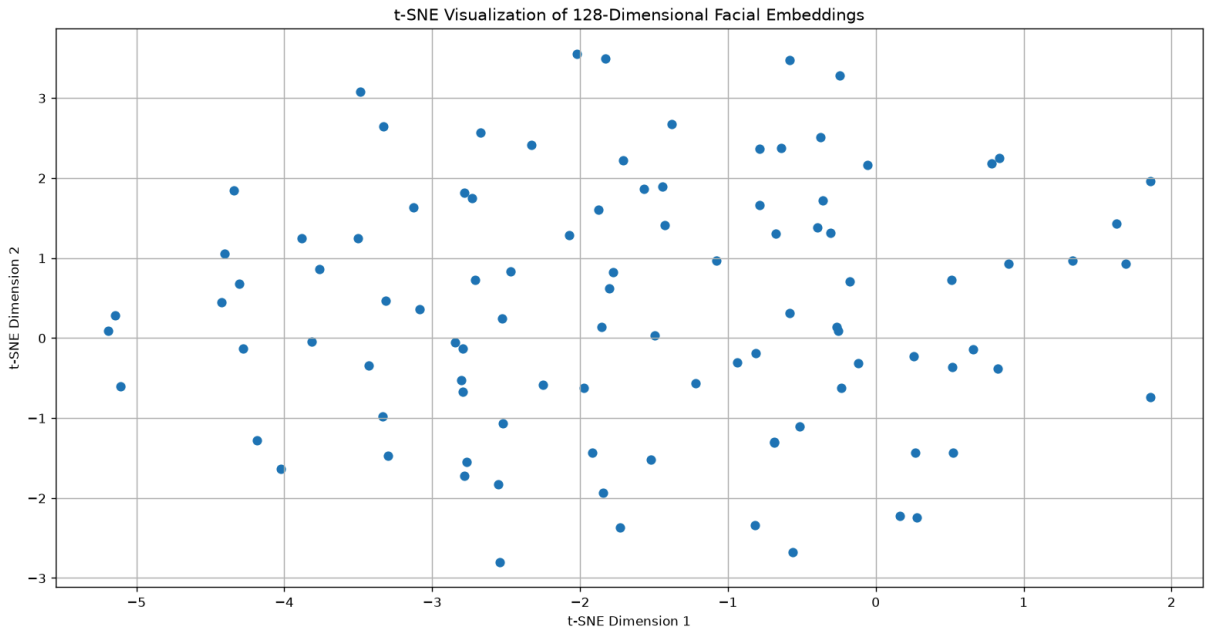
10. t-SNE Visualization of Facial Embeddings

Python Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.manifold import TSNE

np.random.seed(10)

data = np.random.rand(100, 128)

tsne = TSNE(

    n_components=2,

    random_state=42,

    perplexity=30
```

```
)
data_2d = tsne.fit_transform(data)
plt.figure(figsize=(8, 6))
plt.scatter(
    data_2d[:, 0],
    data_2d[:, 1]
)
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("t-SNE Visualization of 128-Dimensional Facial Embeddings")
plt.grid(True)
plt.show()
```

Output:



Interpretation

t-SNE converts the 128-dimensional facial embeddings into two dimensions so that they can be visualized on a scatter plot. Points that are close together in the original high-dimensional space tend to appear close together in the t-SNE visualization.

Why t-SNE is preferred over PCA for visualization?

PCA:

- Finds linear patterns in the data.
- Mainly preserves overall/global variance.
- May not clearly show complex clusters.

t-SNE:

- Captures non-linear relationships.
- Preserves local neighborhood structure.
- Often produces clearer visual clusters.

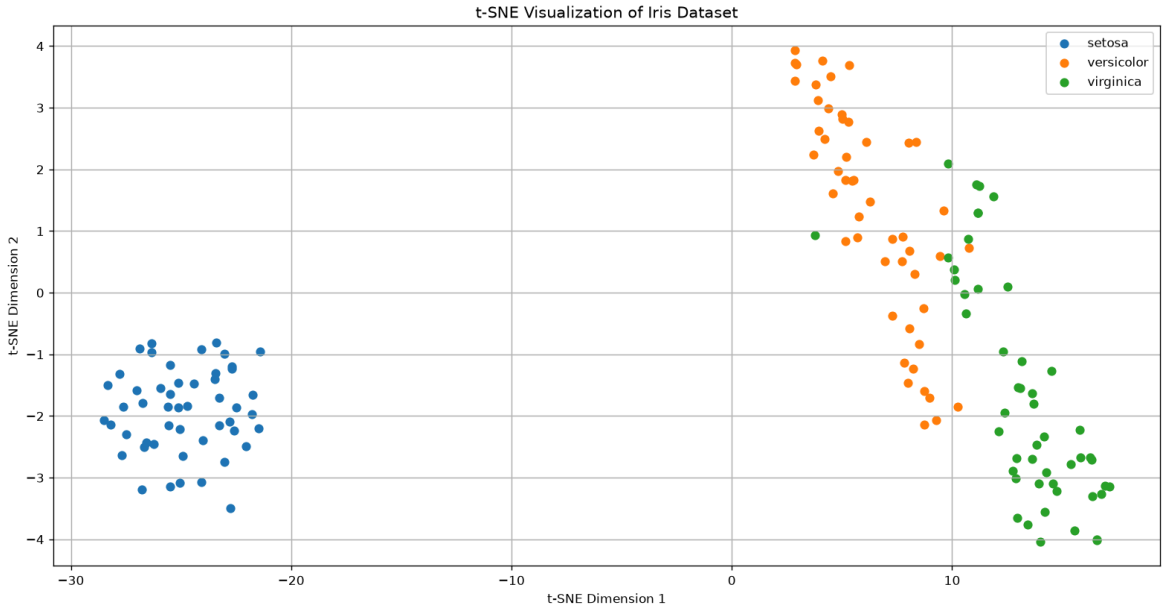
11. t-SNE on Iris Dataset

Python Program:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.manifold import TSNE

iris = load_iris()

X = iris.data

y = iris.target

tsne = TSNE(

    n_components=2,

    random_state=42,

    perplexity=30

)
```

```
X_tsne = tsne.fit_transform(X)
plt.figure(figsize=(8, 6))
for species in range(3):
    plt.scatter(
        X_tsne[y == species, 0],
        X_tsne[y == species, 1],
        label=iris.target_names[species]
    )
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("t-SNE Visualization of Iris Dataset")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Interpretation

The t-SNE plot reduces the four Iris features into two dimensions. The three species are shown using different colors. Setosa is generally well separated from the other two species, while Versicolor and Virginica may show some overlap.

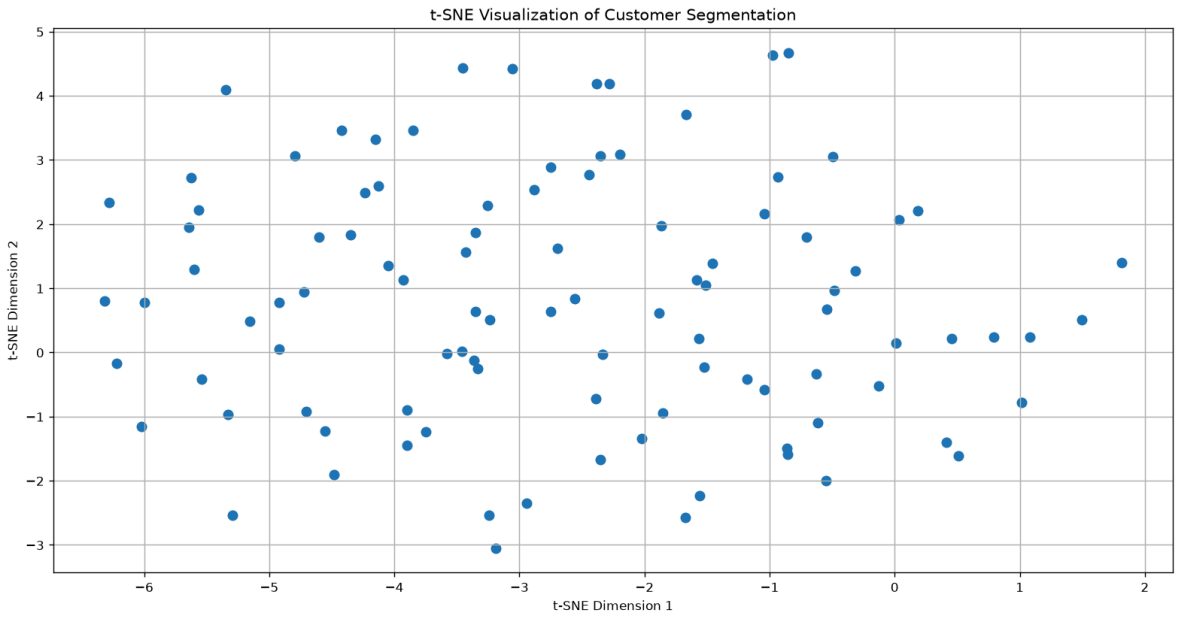
12. Customer Segmentation using t-SNE

Python Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.manifold import TSNE

from sklearn.preprocessing import StandardScaler

np.random.seed(42)

data = np.random.rand(100, 40)

data_scaled = StandardScaler().fit_transform(data)

tsne = TSNE(

    n_components=2,
```

```
    random_state=42,
    perplexity=30
)
data_tsne = tsne.fit_transform(data_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(
    data_tsne[:, 0],
    data_tsne[:, 1],
    s=50
)
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("t-SNE Visualization of Customer Segmentation")
plt.grid(True)
plt.show()
```

Output:



Interpretation

The t-SNE visualization reduces the 40 behavioral variables to two dimensions. Customers that have similar behavioral patterns tend to appear close together, while dissimilar customers appear farther apart.

Cluster Analysis

If clearly separated groups of points appear in the plot, this indicates distinct customer clusters. If the points are heavily mixed without clear separation, distinct customer groups are not strongly evident.

13. UMAP vs PCA for Medical Image Features

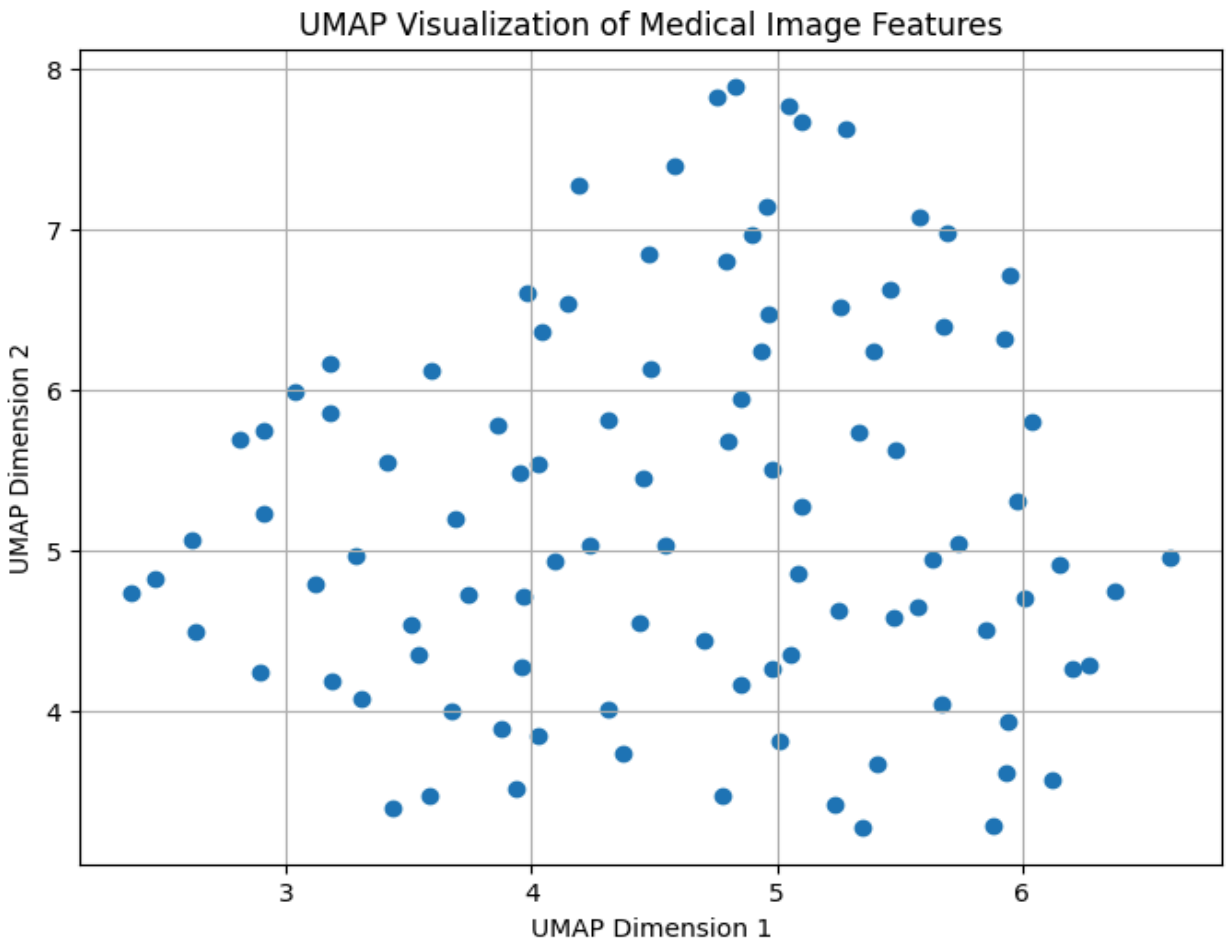
Python Program

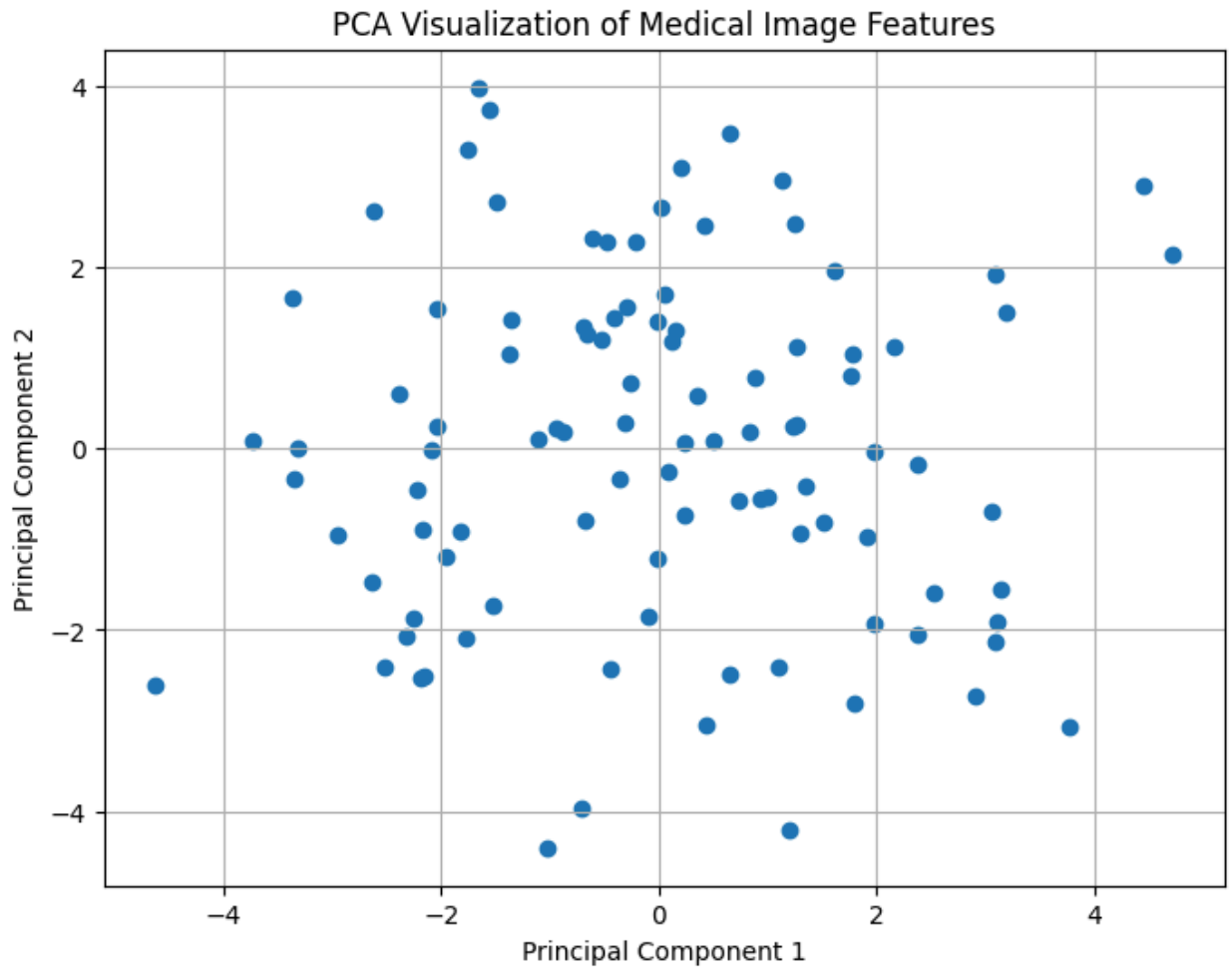
```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

import umap
```

```
np.random.seed(42)
data = np.random.rand(100, 100)
data_scaled = StandardScaler().fit_transform(data)
umap_model = umap.UMAP(
    n_components=2,
   random_state=42
)
umap_result = umap_model.fit_transform(data_scaled)
pca = PCA(n_components=2)
pca_result = pca.fit_transform(data_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(
    umap_result[:, 0],
    umap_result[:, 1]
)
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.title("UMAP Visualization of Medical Image Features")
plt.grid(True)
plt.show()
plt.figure(figsize=(8, 6))
plt.scatter(
    pca_result[:, 0],
```

```
    pca_result[:, 1]
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization of Medical Image Features")
plt.grid(True)
plt.show()
print("PCA Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

Output:






```
PCA Explained Variance Ratio:
[0.03708    0.03586183]
```

Interpretation

UMAP:

UMAP reduces the 100 medical image features to two dimensions while preserving important local relationships between similar observations. It can reveal clusters and complex non-linear patterns.

PCA:

PCA also reduces the 100 features to two dimensions, but it mainly captures linear relationships and the directions with the greatest variance.

Comparison

UMAP	PCA
Non-linear dimensionality reduction	Linear dimensionality reduction
Preserves local structure well	Preserves overall variance
Often reveals clearer clusters	Easier to interpret
Good for complex image data	Fast and simple

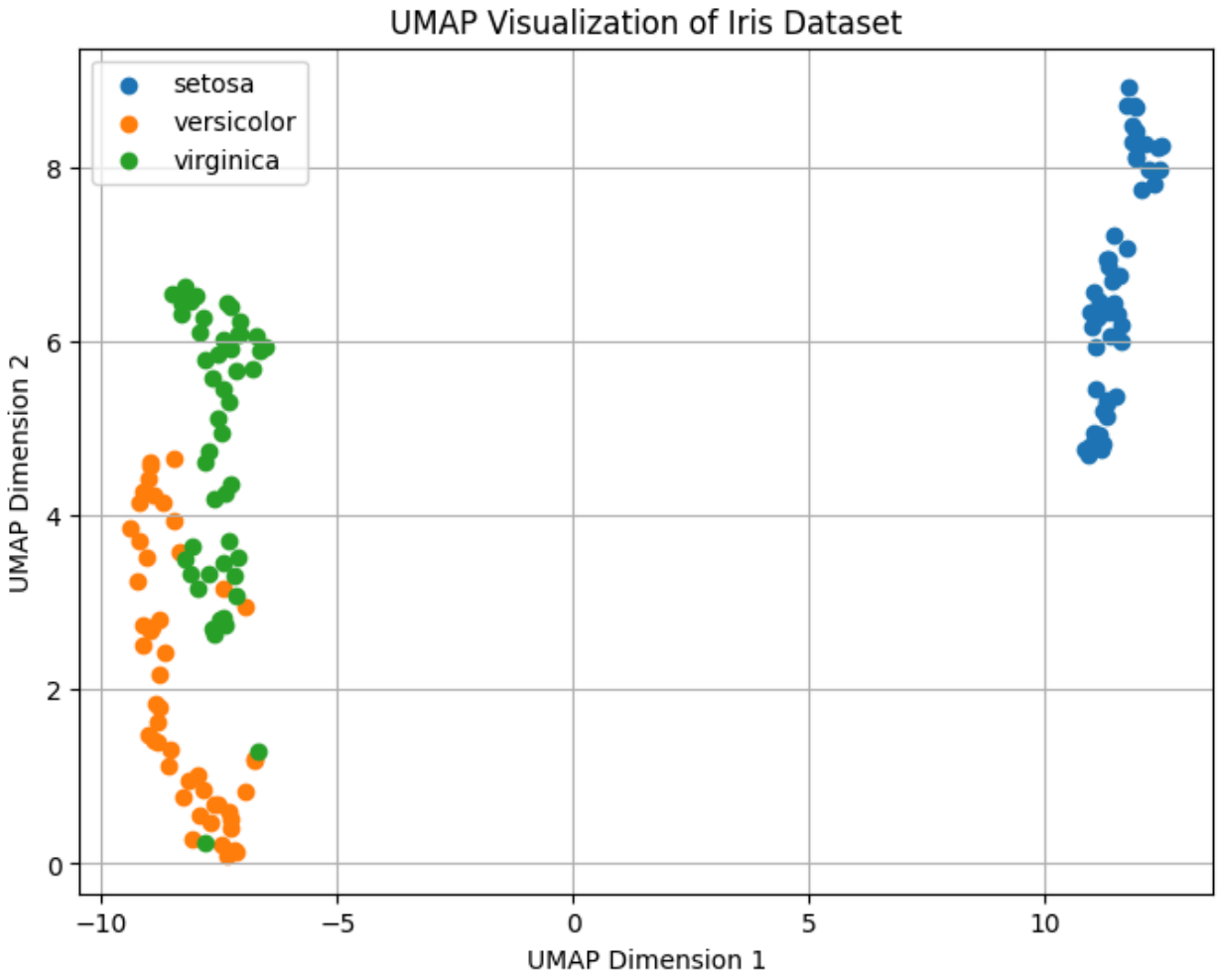
14. UMAP on Iris Dataset

Python Program:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
import umap
iris = load_iris()
X = iris.data
y = iris.target
X_scaled = StandardScaler().fit_transform(X)
umap_model = umap.UMAP(
    n_components=2,
    random_state=42
)
X_umap = umap_model.fit_transform(X_scaled)
plt.figure(figsize=(8, 6))
```

```
for species in range(3):
    plt.scatter(
        X_umap[y == species, 0],
        X_umap[y == species, 1],
        label=iris.target_names[species]
    )
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.title("UMAP Visualization of Iris Dataset")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Interpretation

UMAP reduces the four Iris features into two dimensions. Different colors represent the three species: Setosa, Versicolor, and Virginica. Setosa is generally well separated, while Versicolor and Virginica may show some overlap.

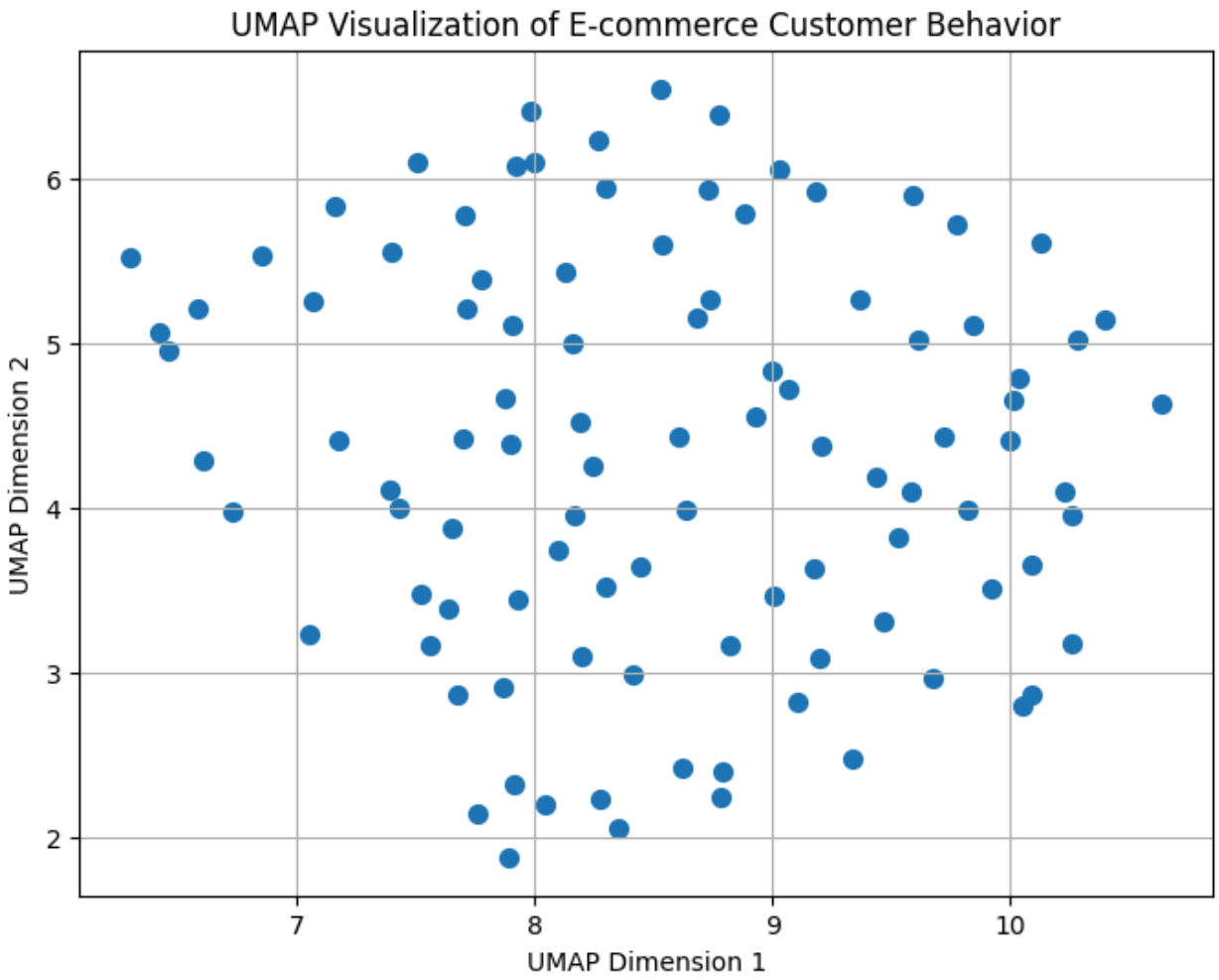
15. UMAP for E-commerce Customer Behavior

Python Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

import umap
```

```
np.random.seed(42)
data = np.random.rand(100, 60)
umap_model = umap.UMAP(
    n_components=2,
    random_state=42
)
data_umap = umap_model.fit_transform(data_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(
    data_umap[:, 0],
    data_umap[:, 1],
    s=50
)
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.title("UMAP Visualization of E-commerce Customer Behavior")
plt.grid(True)
plt.show()
```

Output:



Interpretation

UMAP reduces the 60 purchasing-behavior attributes to two dimensions. Customers with similar purchasing behavior are placed closer together, while customers with different behavior are placed farther apart.

Identifying Customer Clusters

Distinct groups of closely located points in the UMAP plot indicate potential customer clusters. For example, one cluster may represent customers with similar purchasing patterns, while another cluster may represent customers with different behavior.